

v48 독립 재현 명세서 (LG Aimers 9기 Phase 2 · 제구 성공 확률 예측 · 리더보드 1224점 제출물)

이 문서는 원본 데이터(train.csv, test.csv, sample_submission.csv, trackman_history.csv)와 이 문서만 가진 사람이, 기존 코드 없이 v48 파이프라인을 새로 구현해 같은 예측을 만들 수 있도록 쓴 명세서다. 모든 수식·상수·조건·예외 처리는 실제 제출물(submit_v48_lg30.zip, md5 7ed1e400)의 추론 코드와 그 모델을 만든 학습 코드를 한 줄씩 따라가며 옮긴 것이다. 함수 이름은 쓰지 않고 계산 자체를 적는다. 예시 코드는 모두 이 문서 안에서 완결된다(외부 프로젝트 파일 import 없음).

재현 목표와 허용 오차. 트리 모델 중 LightGBM(CPU)은 같은 라이브러리 버전·스레드 수·시드에서 비트 단위로 동일하게 재현된다. CatBoost(GPU)와 신경망(GPU)은 GPU 비결정성 때문에 파일은 달라지며, 실제로 같은 절차로 다시 학습했을 때 최종 예측의 행당 평균 절대 차이는 0.00025, 최대 0.0016, 상관 0.99997이었다(리더보드 점수 영향 $\approx \pm 0.2$ 점). 따라서 "동일한 예측"의 판정 기준은 § 13의 체크포인트(형상·통계값·샘플 5행 예측·평균)를 만족하고 행당 평균 절대 차이가 $1e-3$ 이하인 것으로 둔다.

0. 전체 그림



예측 대상은 4클래스(성공/역방향 실패/가운데 실패/기타 실패) 확률 중 **클래스 0(성공) 확률**이다. 이진 문제를 4클래스로 푸는 이유는 학습 데이터의 누적 지표에서 실패 유형을 복원할 수 있기 때문이며(§ 4), 최종 출력은 성공 확률 하나다.

1. 데이터 파일과 컬럼

1.1 train.csv (1,475,092행 × 49컬럼) / test.csv (평가 서버: 253,507행 × 48컬럼, 배포본은 5행 샘플)

한 행 = 한 투구 직전의 상태. 두 파일의 입력 컬럼 구조는 같고 train에만 정답 control_success가 있다. 이 파이프라인이 실제로 읽는 컬럼과 역할:

컬럼	값	파이프라인에서의 역할
row_id	문자열(TRAIN_0000001..., T EST_000001...)	제출 매칭 키. 피쳐로는 사용하지 않음
season	2019~2024 (test는 2025)	시즌 축. 이전 시즌 통계 조회 키, 피쳐 season (트리), season_rel (NN)
game_month, game_dayofweek	월, 요일(월=0...일=6)	피쳐; 경기 분할 키; LG 체제 판정(2023년 5월)
inning, top_bottom	이닝, T/B	피쳐; 홈팀 판정(초=T이면 투수 팀이 홈)
game_type	R (KBO 1군 정규시즌) / F (퓨처스 2군)	피쳐, 레벨 기준(리그×경기유형 평균), 후처리 조건
balls_before, strikes_before, outs_before	카운트	피쳐; count_state = balls*3+strikes; 3-0 규칙
run_top_before, run_bot_before, run_total_before, score_diff_home, score_diff_pitcher_team	점수	피쳐
runner_on_1b/2b/3b, num_runners_on, base_state	주자; base_state ∈ { __, 1_, __, 2_, __3, 12_, 1_3, _23, 123 }	피쳐(base_state는 0~7 정수 코드)
home_win_expectancy, away_win_expectancy (0~100), li (0~10.83)	기대승률, 중요도	피쳐(원값 그대로)
pitcher_id (792명), batter_id (830명)	익명 정수 ID(예: 21813)	통계 키, CatBoost 범주형, NN 임베딩, LightGBM에서는 정수 수치 피쳐
pitcher_hand, batter_hand	1(좌) / 2(우) — 1이 좌투·좌타 (Trackman Left 비율 0.251 vs 코드 1 비율 0.259로 확인)	피쳐, 플레툰 통계
pitcher_team_id, batter_team_id	{12,13,...,23,25} 13개 값	피쳐(범주형/임베딩), 팀 13 = 이 데이터의 제공 구단(LG): F 행은 100%가 팀 13 관여 경기
asof_pitcher_n	해당 투구 직전까지 투수 누적 투구 수(2019년부터 통산, 시즌 경계에서 리셋 없음)	시즌 분해의 핵심 (§ 3.3)
asof_pitcher_success_rate, _reverse_rate, _middle_rate, _ball_rate, _strike_rate, _fastball_rate, _breaking_rate, _offspeed_rate	누적 비율(분모는 모두 asof_pitcher_n)	피쳐 + 시즌 분해 + 학습 라벨 복원 (§ 4)
asof_pitcher_prev1/3/5_game_success_rate, prev1/3/5_game_middle_rate	직전 1/3/5경기 비율	피쳐(폼)
asof_batter_n, asof_batter_success_rate, asof_batter_middle_rate	타자 누적	피쳐 + 시즌 분해
asof_pitcher_pitchmix_n	구종 표본 수	사용하지 않음
control_success (train만)	1 성공 / 0 실패	정답

결측: 표본 0인 행의 rate는 NaN(train에서 asof_pitcher_success_rate 0.05%, prev1 1.98%, asof_batter_success_rate 0.06%). 트리 모델은 NaN을 그대로 입력하고, NN은 § 8.2의 결측 처리 규칙을 따른다.

1.2 trackman_history.csv (1,793,078행 × 30컬럼, 2019~2024, 5,980경기, 투수 ID 906개)

사용 컬럼: season, game_month, game_dayofweek, trackman_game_id(예 20190329-Gocheok-1), pitch_no, inning, top_bottom(Top/Bottom), balls_before, strikes_before, outs_before, pitcher_trackman_id, batter_trackman_id, pitcher_hand/batter_hand(Left/Right), tagged_pitch_type(Fastball, Slider, Curveball, ChangeUp/Changeup, Splitter, Sinker, Cutter, ...), pitch_type_group(fastball/breaking/offspeed/other), rel_speed, spin_rate, induced_vert_break, horz_break, extension, rel_height, rel_side, zone_speed. 나머지(game_date, pitch_of_pa, auto_pitch_type, 팀명 등)는 사용하지 않는다. 메인 데이터와 직접 키가 없으므로 § 6의 지문 매칭으로 투수 ID 대응표를 만든다.

1.3 sample_submission.csv

row_id, control_success 2컬럼. 제출 파일은 이 파일의 row_id 순서를 그대로 따른다(§ 11.6).

1.4 데이터 사실 (구현 후 확인용)

- 시즌별 행 수: 2019 237,413 / 2020 244,087 / 2021 247,088 / 2022 247,472 / 2023 245,525 / 2024 253,507. game_type: R 1,314,088 / F 161,004.
- 시즌별 성공률(리그 평균): 2019 0.56467 / 2020 0.53271 / 2021 0.53276 / 2022 0.52892 / 2023 0.49996 / 2024 0.48610, 전체 0.52377.
- 시즌×경기유형 성공률: R 2019 0.54949, 2020 0.52692, 2021 0.51276, 2022 0.50369, 2023 0.50312, 2024 0.48971 / F 2019 0.68925, 2020 0.58777, 2021 0.70384, 2022 0.70875, 2023 0.47290, 2024 0.45928.
- 팀 13 관여 R 투구 260,983개(전체 R의 19.9%), F 투구 161,004개.

2. 정렬과 시간 순서 기준

- train.csv의 파일 순서를 그대로 시간 순서로 쓴다.** 어떤 정렬도 하지 않는다. 확인된 사실: row_id가 파일 순서로 단조 증가하고, season이 파일 순서로 비감소이며, 같은 투수의 연속 행 사이에서 asof_pitcher_n이 정확히 1씩 증가한다(100%). 즉 파일 순서 = 투구 시간 순서이고, 이 성질이 § 4(라벨 복원)와 § 7.7(경기 분할)에서 사용된다.
- "이전 시즌"의 정의.** 어떤 행(시즌 y)에 대해 학습 통계에서 가져오는 값은 항상 시즌 < y의 합계다. 시즌 내 정보는 행 자신의 asof_*에서 산술로 복원한다(§ 3.3). 테스트 행(2025)은 2019~2024 전체 합계를 쓴다.
- 시즌 격자.** 이전 시즌 합계는 (개체, 시즌) 격자를 2019부터 2026까지 만들어 계산한다(상한 2026은 코드 상수 MAX_SEASON; 2025·2026 열은 2024까지의 누적을 담은 빈 열이다).
- 테스트 시즌(2025)의 미지 값 처리.** 리그 평균(성공률 등) 같은 시즌별 참조값은 "마지막 학습 시즌(2024)의 값을 그대로 사용"(지속 예측)한다. 구체적 위치는 § 7.4, § 8.4, § 9에 명시한다.
- Trackman 시간축.** 투수별 Trackman 프로파일은 시즌별로 만들고 행 시즌 미만 시즌만 (투구 수 가중) 평균한다(§ 8.6).

3. train / validation / test 구성

- 최종 모델(v48)은 검증 분할 없이 2019~2024 학습 행 1,475,092개 전부로 학습한다. 시즌별 가중치·시즌 상대 위치 피처만 다르다(§ 10).
- 개발 단계에서 쓴 검증은 "시즌 2024를 검증, 2019~2023을 학습"하는 시간 분할이었고, 그때는 통계 테이블도 2023년까지의 행으로만 만들었다. v48 재현에는 필요 없다.
- 테스트 = 2025 시즌 투구 253,507행(평가 서버가 data/test.csv로 제공; F 30,010행, R 223,497행 중 팀 13 관여 44,768행, 그중 3-0 카운트 590행).

3.1 각 행이 볼 수 있는 정보의 범위 (학습·추론 공통)

- 행 자신의 48개 컬럼(그중 § 1.1 표에서 "사용"으로 표시된 것).
- 학습 통계 테이블에서 행 시즌 미만 시즌의 (투수·타자·투타조합·역할·Trackman) 합계/평균.
- 리그 참조값: 행 시즌 - 1의 시즌 × 경기유형 평균 등(2019 행은 2019 자신의 값, 2025 행은 2024 값).
- 학습 행이라도 자기 행의 정답이나 같은 시즌 다른 행의 정답을 피처에 넣지 않는다(시즌 내 정보는 asof 컬럼만). 단, 학습 라벨(§ 4)과 학습 통계(§ 7)를 만들 때는 학습 데이터 안에서 미래 행을 참조한다(주최측이 허용한 범위, § 14).

4. 타깃 정의

4.1 최종 예측 대상

$\text{control_success} \in \{0,1\}$ 의 확률. 평가 지표 $\text{score} = 100000 \times (1 - \text{mean}((p-y)^2) / (r(1-r)))$, r = 테스트 성공률.

4.2 학습용 4클래스 라벨 (트리·NN 공통)

클래스 0 성공, 1 역방향 실패(reverse), 2 가운데/위험 코스 실패(middle), 3 기타 실패(far). 각 학습 행의 지표는 같은 투수의 다음 행 누적값과의 차이로 복원한다.

```
입력: train 행 전체(파일 순서), 컬럼 pitcher_id, asof_pitcher_n(=n), 8개 rate 컬럼, control_success
1. 행에 원래 순번 _ord 부여 후 (pitcher_id, _ord)로 안정 정렬 (같은 투수 안에서는 파일 순서 유지)
2. 각 rate 컬럼 c에 대해 cum = nan_to_num(rate_c × n) # 이 투구 이전까지의 누적 건수 (NaN→0)
   cum_next = 다음 행의 cum (배열을 -1만큼 roll)
   같은 투수의 다음 행이 있으면 ind_c = round(cum_next - cum) (0 또는 1), 없으면 NaN
3. 각 투수의 마지막 행(다음 행 없음) 보정:
   - s(성공): ind = 그 행의 control_success
   - rev, mid: control_success==1이면 0, 아니면 clip(rate_c / clip(1 - success_rate, 1e-3, ∞), 0, 1) (조건부 기대값; rate 절측은 0)
   - ball, strike, fb, br, os: ind = 그 행의 rate 값(절측은 0)
4. 원래 순번으로 되돌림
```

여기서 rate 컬럼과 짧은 이름: success→s, reverse→rev, middle→mid, ball→ball, strike→strike, fastball→fb, breaking→br, offspeed→os. 타자용은 asof_batter_success_rate→s, asof_batter_middle_rate→mid를 batter_id, asof_batter_n 기준으로 같은 절차로 복원하되, 3단계의 mid 보정에서 쓰는 성공률 sr은 타자의 성공률이 아니라 상수 0.52다(원 구현이 타자 프레임에 asof_pitcher_success_rate가 없을 때 0.52로 대체하기 때문). 즉 타자 마지막 행: $\text{mid} = 0 \text{ if } y==1 \text{ else } \text{clip}(\text{rate_mid} / (1-0.52), 0, 1)$. 이 값

은 § 7.2의 `bs.mid`를 거쳐 `b_prior_mid_rate`, `b_cur_mid_rate`, `b_chg_mid`에 들어가므로 반드시 상수를 써야 한다.

4클래스 라벨: `far = clip(1 - s - rev - mid, 0, 1)`; `y4 = argmax([s, rev, mid, far])` (동률이면 앞 클래스). 복원된 `s`는 train의 `control_success`와 100% 일치하고 전부 0/1이다. 체크포인트: `y4` 분포 = [772,603 / 337,950 / 170,320 / 194,219]; 실패 클래스 비율 `fshare` = [0.4810751, 0.2424522, 0.2764727] (클래스 1,2,3을 실패 합으로 나눈 값; § 9에서 사용).

4.3 NN 보조 라벨 (NN만)

- 볼/스트라이크 3클래스: `argmax([ball, strike, clip(1-ball-strike,0,1)])` → 분포 [545,138 / 654,129 / 275,825].
- 구종군 3클래스: `argmax([fb, br, os])` → 분포 [798,954 / 436,195 / 239,943].

5. 전처리 규칙 (요약; 세부는 각 단계에)

- CSV는 `encoding='utf-8-sig'`로 읽는다. 컬럼 형 변환은 하지 않는다(정수/실수/문자열 그대로).
- `base_state` 문자열 → 정수 코드: `__ 0, 1__ 1, _2_ 2, __33, 12_ 4, 1_35, _236, 1237`.
- `count_state = balls_before*3 + strikes_before` (0~11).
- `is_top = (top_bottom=='T')`, `game_type_F = (game_type=='F')`, `platoon_same = (pitcher_hand==batter_hand)`, `home_team = pitcher_team_id if top_bottom=='T' else batter_team_id`.
- 트리 모델: 수치 스케일링 없음, NaN 그대로. LightGBM 범주형 = `pitcher_team, batter_team, base_state, count_state`, `pitcher_id/batter_id`는 정수 수치 피쳐. CatBoost 범주형 = 위 4개 + `pitcher_id, batter_id`(정수로 캐스팅).
- NN: 수치 117개를 분위수 변환(정규 분포 출력) + 결측 지시자, 범주 7개(무ID 모델은 5개)를 임베딩 (§ 8.2, § 10.4).
- 0으로 나누기: 분모가 0이면 NaN(단, 아래 각 식에서 `max(den,1)`로 명시된 곳은 그 규칙). NumPy 경고는 무시.

6. [1단계] Trackman ↔ 메인 데이터 투수 ID 대응표

목적: `pitcher_id`(메인) ↔ `pitcher_trackman_id`(Trackman)를 추정해 투수별 Trackman 프로필을 이전 시즌 피쳐로 쓰기 위함. (타자 대응표도 같은 방식으로 만들지만 v48 피쳐에서는 투수 대응표만 사용한다.)

6.1 메인 데이터 경기 분할

train 행을 파일 순서대로 보며, 키 (`season, game_month, game_dayofweek, teamA, teamB, game_type`)가 직전 행과 달라지거나 `inning`이 직전 행보다 작아지면 새 경기 시작. 여기서 `teamA = pitcher_team_id if top_bottom=='T' else batter_team_id`, `teamB`는 그 반대(즉 `teamA` = 홈팀). 경기 번호 `gid`는 누적 카운트. 체크포인트: 경기 수 4,868.

6.2 지문(fingerprint) 매칭

- 메인 경기 `g`의 지문 = 각 투구의 튜플 (`inning, top_bottom, balls_before, strikes_before, outs_before, pitcher_hand, batter_hand`) 리스트.

- Trackman은 (trackman_game_id, pitch_no) 정렬 후 `tb = 'T' if top_bottom=='Top' else 'B', p`
`h = 1 if pitcher_hand=='Left' else 2, bh` 동일 규칙으로 같은 튜플 리스트를 만든다.
- 메인 경기 키 (season, game_month, game_dayofweek) 가 같은 Trackman 경기들만 후보. 유사도 = 두 튜플 다중집합의 교집합 크기 / max(두 길이). 최댓값 후보를 매칭으로 채택(동률이면 먼저 나온 것).
- 체크포인트: 유사도 분위수 [min .0, 5% .229, 10% .268, 25% .986, 50%~max 1.0]; 유사도 > 0.9 경기 비율 0.873; 한 Trackman 경기가 유사도>0.8인 메인 경기 2개 이상에 매칭된 경우 1건.

6.3 투수 대응표

유사도 ≥ 0.85 인 경기만 사용. 각 (이닝, 초/말) 반이닝에서 메인 투수가 정확히 1명이고 Trackman 투수도 정확히 1명이면 그 쌍에 **메인 측 투구 수만큼** 표를 준다. 메인 투수마다 표가 가장 많은 Trackman ID(동률이면 먼저 집계된 쌍)를 대응으로 택하고 `purity = 그 표 / 그 투수의 전체 표`, `votes = 그 표`를 기록한다. 체크포인트: 대응 760/792명, `purity`>0.9 비율 0.996, `votes` ≥ 100 비율 0.787, 한 Trackman ID가 2명 이상에 대응된 경우 1건. (타자: 반이닝 안에서 양쪽 타자 수가 같을 때 순서대로 1표씩; 816/830명.)

```
# 6단계 예시 (독립 실행 가능)
import numpy as np, pandas as pd
from collections import Counter, defaultdict
tr = pd.read_csv('data/train.csv', encoding='utf-8-sig'); tm = pd.read_csv('data/trackman_history.csv', encoding='utf-8-sig')
tA = np.where(tr.top_bottom=='T', tr.pitcher_team_id, tr.batter_team_id); tB = np.where(tr.top_bottom=='T', tr.batter_team_id, tr.pitcher_team_id)
gk = tr.season.astype(str)+'_'+tr.game_month.astype(str)+'_'+tr.game_dayofweek.astype(str)+'_'+pd.Series(tA).astype(str)+'_'+pd.Series(tB).astype(str)+'_'+tr.game_type
tr['gid'] = ((gk != gk.shift()) | (tr.inning < tr.inning.shift())).cumsum()
tm = tm.sort_values(['trackman_game_id', 'pitch_no']).reset_index(drop=True)
tm['tb'] = np.where(tm.top_bottom=='Top', 'T', 'B'); tm['ph'] = np.where(tm.pitcher_hand=='Left', 1, 2); tm['bh'] = np.where(tm.batter_hand=='Left', 1, 2)
fp = lambda g, tb, ph, bh: list(zip(g.inning, g[tb], g.balls_before, g.strikes_before, g.outs_before, g[ph], g[bh]))
main_games = {gid: (g.season.iloc[0], g.game_month.iloc[0], g.game_dayofweek.iloc[0], fp(g, 'top_bottom', 'pitcher_hand', 'batter_hand')) for gid, g in tr.groupby('gid')}
tm_games = {gid: (g.season.iloc[0], g.game_month.iloc[0], g.game_dayofweek.iloc[0], fp(g, 'tb', 'ph', 'bh')) for gid, g in tm.groupby('trackman_game_id')}
by_key = defaultdict(list)
for gid, (s, m, d, f) in tm_games.items(): by_key[(s, m, d)].append(gid)
matches = {}
for gid, (s, m, d, fa) in main_games.items():
    ca = Counter(fa); best, bs = None, 0
    for c in by_key.get((s, m, d), []):
        fb = tm_games[c][3]; sim = sum((ca & Counter(fb)).values()) / max(len(fa), len(fb))
        if sim > bs: best, bs = c, sim
    matches[gid] = (best, bs)
votes = Counter(); tmi = tm.set_index('trackman_game_id')
for gid, (tg, sim) in matches.items():
    if sim < 0.85: continue
    a = tr[tr.gid==gid]; b = tmi.loc[[tg]]
    for (inn, tb), ga in a.groupby(['inning', 'top_bottom']):
        gb = b[(b.inning==inn) & (b.tb==tb)]; pa = ga.pitcher_id.unique(); pb = gb.pitcher_trackman_id.unique()
        if len(pa)==1 and len(pb)==1: votes[(pa[0], pb[0])] += len(ga)
pm = defaultdict(Counter)
for (p, q), v in votes.items(): pm[p][q] += v
pmap = pd.DataFrame([{'pitcher_id': p, 'tm_id': cnt.most_common(1)[0][0], 'votes': cnt.most_common(1)[0][1], 'purity': cnt.most_common(1)[0][1]/sum(cnt.values())} for p, cnt in pm.items()])
```

7. [2단계] 학습 통계 테이블 (train 전체 2019~2024로 1회 계산)

아래 표들이 추론 시에도 그대로 실린다(stats.pkl). 추론에 실제로 쓰이는 것은 **ps, bs, pbs, league, L_gt, L_all, ps2, bs2, role, tm** 10개다(FM_gt, team_p, team_b, endform, src_eff는 만들어지지만 v48 추론 경로에서는 읽지 않는다).

7.1 ps — 투수×시즌 합계 (2,260행 × 32열)

§ 4.2의 복원 지표(pind: s, rev, mid, ball, strike, fb, br, os)를 행마다 만든 뒤, 아래 행 단위 값을 (pitcher_id, season) 으로 **합산**한다.

열	행 단위 값
n	1
s, rev, mid, ball, strike, fb, br, os	복원 지표 값
n_F	game_type=='F'
s_F	n_F × control_success
n_LG	game_type=='R' 이고 (pitcher_team_id==13 또는 batter_team_id==13)
s_LG	n_LG × control_success
n_L	batter_hand==1
s_L	n_L × control_success
n_2k	strikes_before==2
s_2k	n_2k × control_success
n_00	balls_before==0 이고 strikes_before==0
s_00	n_00 × control_success
n_late	inning>=7
s_late	n_late × control_success
n_L2k	batter_hand==1 이고 strikes_before==2
s_L2k	n_L2k × control_success
rev_L, mid_L	n_L × rev, n_L × mid
s_fb, s_br, s_os	fb×control_success, br×..., os×...
n_F23, s_F23	n_F × (season>=2023), s_F × (season>=2023)

체크포인트: 합계 n 1,475,092 / s 772,603 / n_F 161,004 / n_LG 260,983. (참고: 최신 학습 코드는 n_new=(season>=2023), s_new 2열을 더 만들지만 v48 제출물의 표에는 없고 어떤 피처에도 쓰이지 않는다. 만들어도 결과는 같다.)

7.2 bs — 타자×시즌 합계 (2,393행 × 7열): n=1, s, mid (타자 복원 지표), n_F, s_F.

7.3 pbs — 투수×타자×시즌 (150,624행): 키 pb_key = pitcher_id×100000 + batter_id, n=1, s=control_success 합.

7.4 리그 참조값

- league: 시즌별 league_rate (성공률 평균), league_n (행 수) — § 1.4의 값.
- L_gt: 시즌×game_type 성공률 표 (§ 1.4). L_all: 시즌별 성공률(= league_rate).
- max_season = 2024.

7.5 ps2 — 투수×시즌 잔차 합계 (2,260행 × 23열)

행 잔차 res = control_success - L_gt[season, game_type]. 실패유형 잔차: rev_res = rev - mean(rev | season, game_type), mid_res, far_res 동일(far = clip(1 - y - rev - mid, 0, 1), 평균은 시즌×경기유형 그룹 평균). 상황 지시자: ahead = strikes_before > balls_before, behind = balls_before > strikes_before, two_k = strikes_before==2, three_b = balls_before==3, vsL = batter_hand==1, home = top_bottom=='T', isF = game_type=='F'. 합산 열: n=1, res, n_R=-isF, res_R=-isF×res, n_F, res_F, n_L

=vsl, res_L, rev_res, mid_res, far_res, n_ahead, res_ahead, n_behind, res_behind, n_2k, res_2k, n_3b, res_3b, n_home, res_home.

7.6 bs2 — 타자×시즌 잔차 합계 (2,393행 × 9열): n, res, mid_res (= 타자 복원 mid - 시즌×경기 유형 평균 mid), n_vsL (= pitcher_hand==1), res_vsL, n_2k, res_2k.

7.7 role — 투수 역할 (2,260행): § 6.1과 같은 경기 분할 gid를 만들고, 각 (경기, 초/말)의 첫 행 투수를 선발로 본다. 투수별 경기 단위로 n(투구 수), start(선발 여부 최대), inn_first(최소 이닝), inn_last(최대 이닝)을 만든 뒤 (투수, 시즌)별로 games=경기 수, starts=start 평균(선발 비율), ppg=n 평균, inn_first·inn_last=평균.

7.8 tm — 투수 Trackman 프로파일 (2,305행 × 36열)

§ 6의 대응표에서 purity > 0.9 이고 votes >= 30 인 투수만 사용(760명 중 **701명**; 이 필터로 § 6.3의 '한 Trackman ID가 2명에 대응된 1건'도 제거되므로 Trackman 행이 두 투수에 중복 귀속되는 일은 없다). Trackman 행을 pitcher_trackman_id → pitcher_id로 붙이고, 파생: ChangeUp = tagged_pitch_type ∈ {ChangeUp, Changeup}, 구종별 지시자(Fastball, Slider, Curveball, Splitter, Sinker, Cutter는 tagged_pitch_type == 이름), minor = trackman_game_id가 정규식 'Minor|Futures|Test'를 포함, is_fb = pitch_type_group=='fastball'. (투수, 시즌)별로: - tm_n = 행 수(측정값이 NaN인 행도 포함); 8개 지표(rel_speed, spin_rate, induced_vert_break, horz_break, extension, rel_height, rel_side, zone_speed)의 tm_<m>_mean, tm_<m>_std(표본 표준편차 ddof=1) — **평균·표준편차는 NaN을 제외하고 계산**(rel_speed NaN 7,617행, spin_rate NaN 12,465행 등이 있음), 값이 1개뿐이면 std는 NaN; - tm_mix_<구종> 7개 = 구종 지시자 평균; tm_minor_share = minor 평균; - 패스트볼(is_fb) 행만으로 tm_fb_speed(rel_speed 평균), tm_fb_spin, tm_fb_ivb, tm_fb_hb, tm_fb_relh_std(rel_height std), tm_fb_rels_std, tm_fb_speed_std; - tm_rel_consistency_h: (투수, 시즌, pitch_type_group) 그룹마다 n=그룹 행 수(NaN 행 포함), rh=그룹 내 rel_height 표본 std(NaN 제외; 행이 1개이거나 전부 NaN이면 NaN). 투수×시즌으로 $\sum(rh \times n)$ 과 $\sum n$ 을 더할 때 **rh가 NaN인 그룹은 분자에 0으로 들어가지만 그 그룹의 n은 분모에 그대로 남는다**(pandas sum의 NaN 무시 동작). 결과 = $\sum(rh \times n) / \sum n$. tm_rel_consistency_s는 rel_side로 동일. (NaN 그룹은 969개 — 대부분 'other' 구종군 — 이 규칙을 지키지 않으면 2,305행 중 950행의 값이 달라진다.)

7.9 기타(추론 미사용): FM_gt(시즌×경기유형 실패유형 평균), team_p/team_b(시즌×팀 성공률), endfrom(시즌 말 품), src_eff(시즌별 LG/비LG/F 라벨 소스 효과).

```
# 7.1~7.4 예시: 복원 지표와 투수 시즌 합계
RATE_P = {'asof_pitcher_success_rate': 's', 'asof_pitcher_reverse_rate': 'rev', 'asof_pitcher_middle_rate': 'mid', 'asof_pitcher_ball_rate': 'ball',
          'asof_pitcher_strike_rate': 'strike', 'asof_pitcher_fastball_rate': 'fb', 'asof_pitcher_breaking_rate': 'br', 'asof_pitcher_offspeed_rate': 'os'}
def recover(df, id_col, n_col, rate_cols):
    d = df[[id_col, n_col, 'control_success']] + list(rate_cols).copy(); d['_ord'] = np.arange(len(d)); d = d.sort_values([id_col, '_ord'])
    n = d[n_col].values.astype(float); same = (d[id_col].shift(-1) == d[id_col]).values; out = {}
    for c, k in rate_cols.items():
        cum = np.nan_to_num(d[c].values * n); out[k] = np.where(same, np rint(np.roll(cum, -1) - cum), np.nan)
    res = pd.DataFrame(out, index=d.index); y = d.control_success.values.astype(float); last = ~same
    for k in res.columns:
        col = res[k].values; c = [c for c, kk in rate_cols.items() if kk == k][0]; r = np.nan_to_num(d[c].values[last])
        if k == 's': col[last] = y[last]
        elif k in ('rev', 'mid'):
            sr = np.nan_to_num(d['asof_pitcher_success_rate'].values[last]) if 'asof_pitcher_success_rate' in d else 0.52
            col[last] = np.where(y[last] == 1, 0.0, np.clip(r / np.clip(1 - sr, 1e-3, None), 0, 1))
        else: col[last] = r
        res[k] = col
    return res.sort_index()
pind = recover(tr, 'pitcher_id', 'asof_pitcher_n', RATE_P)
bind = recover(tr, 'batter_id', 'asof_batter_n', {'asof_batter_success_rate': 's', 'asof_batter_middle_rate': 'mid'}) # 타자: sr=0.52 분기
p = pd.DataFrame({'pitcher_id': tr.pitcher_id, 'season': tr.season, 'n': 1.0}); y = tr.control_success
for k in RATE_P.values(): p[k] = pind[k].values
p['n_F'] = (tr.game_type=='F')*1.0; p['s_F'] = p.n_F*y
p['n_LG'] = ((tr.game_type=='R') & ((tr.pitcher_team_id==13)|(tr.batter_team_id==13)))*1.0; p['s_LG'] = p.n_LG*y
p['n_L'] = (tr.batter_hand==1)*1.0; p['s_L'] = p.n_L*y; p['n_2k'] = (tr.strikes_before==2)*1.0; p['s_2k'] = p.n_2k*y
p['n_00'] = ((tr.balls_before==0)&(tr.strikes_before==0))*1.0; p['s_00'] = p.n_00*y; p['n_late'] = (tr.inning>=7)*1.0; p['s_late'] = p.n_late*y
p['n_L2k'] = ((tr.batter_hand==1)&(tr.strikes_before==2))*1.0; p['s_L2k'] = p.n_L2k*y; p['rev_L'] = p.n_L*p.rev; p['mid_L'] = p.n_L*p.mid
for k in ('fb', 'br', 'os'): p['s_'+k] = p[k]*y
p['n_F23'] = p.n_F*(tr.season>=2023); p['s_F23'] = p.s_F*(tr.season>=2023)
ps = p.groupby(['pitcher_id', 'season']).sum().reset_index()
league = tr.groupby('season').control_success.agg(['mean', 'size']).reset_index(); league.columns = ['season', 'league_rate', 'league_n']
L_gt = tr.groupby(['season', 'game_type']).control_success.mean().unstack(); L_all = tr.groupby('season').control_success.mean()
```

8. [3단계] 행 단위 피처 — 정확한 정의

학습 행과 테스트 행에 **완전히 같은 계산을 적용**한다(학습 행은 자기 시즌 미만의 통계만 보게 되고, 테스트 행은 2024까지 전부를 본다). 아래 표기: $d.<컬럼>$ 은 행 자신의 원본 컬럼, $prior_*$ 는 § 8.1에서 붙인 이전 시즌 합계.

8.1 이전 시즌 합계 붙이기 ("prior 테이블")

어떤 (개체, 시즌) 합계 표 T(열 집합 C)에 대해, 개체 $\times$ 시즌(2019...2026) 격자를 만들어 빈 칸은 0으로 채운 뒤 시즌 축으로 **누적합을 한 시즌 뒤로 민 값**(= 시즌 $< y$ 합)을 $prior_c$ 로 둔다. 추가로 - $last_season_gap = y - (y$ 이전에 $n > 0$ 이었던 마지막 시즌); $last_n, last_s$ = 그 마지막 활동 시즌의 n, s (또는 지정 열); 활동 이력이 없으면 NaN. - $n_seasons = y$ 이전에 $n > 0$ 인 시즌 수. 행에 (id, season) 으로 left-join 한다. join 실패(학습 표에 없는 개체)는 $prior_*$ 와 $n_seasons$ 를 0으로, $last_*$ 는 NaN으로 둔다.

```
def prior_table(tab, id_col, cols, max_season=2026, last_col='s'):
    ids = tab[id_col].unique(); seasons = np.arange(tab.season.min(), max_season + 1)
    grid = pd.MultiIndex.from_product([ids, seasons], names=[id_col, 'season'])
    t = tab.set_index([id_col, 'season']).reindex(grid).fillna(0.0)
    wide = {c: t[c].unstack('season') for c in cols}; out = {}
    for c in cols: out['prior_' + c] = wide[c].cumsum(axis=1).shift(1, axis=1).fillna(0.0)
    n_w = wide['n']; act = n_w.gt(0)
    seas = pd.DataFrame(np.tile(seasons, (len(ids), 1)), index=n_w.index, columns=n_w.columns)
    out['last_season_gap'] = seas - seas.where(act).ffill(axis=1).shift(1, axis=1)
    out['last_n'] = n_w.where(act).ffill(axis=1).shift(1, axis=1)
    out['last_s'] = wide[last_col].where(act).ffill(axis=1).shift(1, axis=1)
    out['n_seasons'] = act.astype(float).cumsum(axis=1).shift(1, axis=1).fillna(0.0)
    return pd.concat({k: v.stack(dropna=False) for k, v in out.items()}, axis=1).reset_index()
```

투수: `prior_table(ps, 'pitcher_id', ['n', 's', 'rev', 'mid', 'ball', 'strike', 'fb', 'br', 'os', 'n_F', 's_F', 'n_L', 's_L', 's_fb', 's_br', 's_os', 'n_F23', 's_F23', 'n_2k', 's_2k', 'n_00', 's_00', 'n_late', 's_late', 'n_L2k', 's_L2k', 'rev_L', 'mid_L'])`. 타자: `prior_table(bs, 'batter_id', ['n', 's', 'mid', 'n_F', 's_F'])`, 열 이름 앞에 b를 붙여 `bprior_n, bprior_s, bprior_mid, bprior_n_F, bprior_s_F, blast_season_gap, blast_n, blast_s, bn_seasons`.

8.2 원본 상황 피처 (31개)

`season, game_month, game_dow(=game_dayofweek), inning, is_top, game_type_F, balls, strikes, count_state, outs, run_top, run_bot, run_total, score_diff_home, score_diff_p(=score_diff_pitcher_team), abs_score_diff=|score_diff_pitcher_team|, r1, r2, r3, n_runners, base_state(코드), home_we=home_win_expectancy, pitcher_we = home_win_expectancy if top_bottom=='T' else away_win_expectancy, li, pitcher_hand, batter_hand, platoon_same, pitcher_team, batter_team, pitcher_id, batter_id`.

원본 asof 피처(18개, 값 그대로): `asof_p_n, asof_p_success_rate, asof_p_reverse_rate, asof_p_middle_rate, asof_p_ball_rate, asof_p_strike_rate, asof_p_fastball_rate, asof_p_breaking_rate, asof_p_ofspeed_rate, asof_p_prev1/3/5_game_success_rate, asof_p_prev1/3/5_game_middle_rate, asof_b_n, asof_b_success_rate, asof_b_middle_rate`.

8.3 투수 시즌 분해 (핵심)

$pn = \text{asof_pitcher_n}(\text{float})$. - $p_prior_n = \text{prior_n}$; $p_cur_n = \max(pn - \text{prior_n}, 0)$ ← 현 시즌 투구 수 (시즌 내 정보의 정확한 복원). - 각 지표 $k \in \{s, \text{rev}, \text{mid}, \text{ball}, \text{strike}, \text{fb}, \text{br}, \text{os}\}$ 에 대해 $\text{tot_k} = \text{nan_to_num}(\text{rate_k} \times pn)$, $\text{cur_k} = \text{tot_k} - \text{prior_k}$; - $p_cur_<k>_rate = \text{cur_k} / \max(p_cur_n, 1)$ ($p_cur_n > 0$ 일 때, 아니면 NaN) - $p_prior_<k>_rate = \text{prior_k} / \max(\text{prior_n}, 1)$ ($\text{prior_n} > 0$ 일 때, 아니면 NaN) - $p_cur_s = \text{nan_to_num}(\text{asof_pitcher_success_rate} \times pn) - \text{prior_s}$ (현 시즌 성공 수; NaN 아님) - $p_prior_F_share = \text{prior_n_F} / \text{prior_n}$ ($\text{prior_n} > 0$), $p_prior_F_rate = \text{prior_s_F} / \text{prior_n_F}$ ($\text{prior_n_F} > 0$), $p_prior_F23_n = \text{prior_n_F23}$, $p_prior_F23_rate = \text{prior_s_F23} / \max(\text{prior_n_F23}, 1)$ ($\text{prior_n_F23} \geq 20$ 일 때만), $p_prior_F23_share = \text{prior_n_F23} / \max(\text{prior_n}, 1)$ ($\text{prior_n} > 0$), $p_prior_R_rate = (\text{prior_s} - \text{prior_s_F}) / (\text{prior_n} - \text{prior_n_F})$ (분모 > 0), $p_prior_vsL_rate = \text{prior_s_L} / \text{prior_n_L}$ (> 0), $p_prior_vsR_rate = (\text{prior_s} - \text{prior_s_L}) / (\text{prior_n} - \text{prior_n_L})$ (> 0), $p_prior_vsHand_rate = \text{vsL}$ if $\text{batter_hand} == 1$ else vsR .

8.4 CTX(문맥 차이) 피쳐 — 축소된 플래툰·2스트라이크 차이

정의 $\text{shr_diff}(s1, n1, s0, n0; K=1000)$: $m1 = s1 / \max(n1, 1)$, $m0 = s0 / \max(n0, 1)$, $ne = 1 / (1 / \max(n1, 1) + 1 / \max(n0, 1))$, 결과 = $(m1 - m0) \cdot ne / (ne + K)$ ($n1 > 0$ 이고 $n0 > 0$ 일 때, 아니면 0). - $\text{isL} = \text{batter_hand} == 1$; $n1 = \text{prior_n_L}$ if isL else $\text{prior_n} - \text{prior_n_L}$, $s1 = \text{prior_s_L}$ if isL else $\text{prior_s} - \text{prior_s_L}$. - $\text{CTX_hand_diff} = \text{shr_diff}(s1, n1, \text{prior_s} - s1, \text{prior_n} - n1)$; $\text{CTX_hand_adj} = \text{CTX_hand_diff} \times ((\text{prior_n} - n1) / \max(\text{prior_n}, 1) \text{ if } \text{prior_n} > 0 \text{ else } 0.5)$. - $d2 = \text{shr_diff}(\text{prior_s_2k}, \text{prior_n_2k}, \text{prior_s} - \text{prior_s_2k}, \text{prior_n} - \text{prior_n_2k})$; $\text{sh2} = \text{prior_n_2k} / \max(\text{prior_n}, 1)$ if $\text{prior_n} > 0$ else 0.28; $\text{CTX_2k_diff} = d2$; $\text{CTX_2k_adj} = d2 \times (1 - \text{sh2})$ if $\text{strikes_before} == 2$ else $-d2 \times \text{sh2}$.

8.5 직전 시즌·시즌 수, 구종별 사전, 타자, 축소 추정

- $p_last_n = \text{last_n}$, $p_last_rate = \text{last_s} / \text{last_n}$ ($\text{last_n} > 0$), $p_last_gap = \text{last_season_gap}$, $p_n_seasons = n_seasons$.
- 구종별: $\text{base} = p_prior_s_rate$; $k \in \{\text{fb}, \text{br}, \text{os}\}$: $n_k = \text{prior_k}$ (그 구종의 이전 시즌 투구 수), $s_k = \text{prior_s_k}$; $p_prior_s_rate_k = s_k / \max(n_k, 1)$ ($n_k \geq 30$ 일 때만); $\text{shr_k} = (s_k + 100 \cdot \text{nan_to_num}(\text{base}, 0.52)) / (n_k + 100)$; $\text{mix_k} = p_cur_k_rate$, NaN이면 $n_k / \max(\text{prior_n}, 1)$ ($\text{prior_n} > 0$) 아니면 NaN → $\text{clip}(\text{nan_to_num}(\text{mix}, 0), 0, 1)$; $p_mix_exp_rate = \sum \text{mix_k} \cdot \text{shr_k} / \sum \text{mix_k}$ ($\sum \text{mix} > 0$), $p_mix_exp_minus_prior = p_mix_exp_rate - \text{base}$.
- 타자: $bn = \text{asof_batter_n}$; $b_prior_n = \text{bprior_n}$, $b_cur_n = \max(bn - \text{bprior_n}, 0)$; $k \in \{s, \text{mid}\}$: $b_cur_<k>_rate = (\text{nan_to_num}(\text{rate_k} \times bn) - \text{bprior_k}) / \max(b_cur_n, 1)$ ($b_cur_n > 0$), $b_prior_<k>_rate = \text{bprior_k} / \max(\text{bprior_n}, 1)$ ($\text{bprior_n} > 0$); $b_last_rate = \text{blast_s} / \text{blast_n}$ ($\text{blast_n} > 0$), b_last_n , b_last_gap , $b_n_seasons$.
- 축소 추정($lp = 0.52$ 고정): $pr = p_prior_s_rate$ (NaN → 0.52); $p_cur_s_shr100 = (\text{nan_to_num}(p_cur_s) + 100 \cdot pr) / (p_cur_n + 100)$, $p_cur_s_shr500$ 동일(500); $p_all_s_shr200 = (\text{nan_to_num}(\text{asof_pitcher_success_rate} \times pn) + 200 \cdot 0.52) / (pn + 200)$; $br = b_prior_s_rate$ (NaN → 0.52), $b_cur_s = \text{nan_to_num}(\text{asof_batter_success_rate} \times bn) - \text{bprior_s}$, $b_cur_s_shr300 = (b_cur_s + 300 \cdot br) / (b_cur_n + 300)$, $b_all_s_shr300 = (\text{nan_to_num}(\text{asof_batter_success_rate} \times bn) + 300 \cdot 0.52) / (bn + 300)$.

8.6 투타 매치업, LG 체제, 역할, Trackman

- 매치업: $pb_key = pitcher_id \times 100000 + batter_id$; § 8.1 방식으로 pbs에서 (pb_key, season) prior_n, prior_s(행에 등장하는 키만 격자로 만들어도 결과 동일). $pb_prior_n, pb_prior_rate = prior_s / \max(prior_n, 1)$ (prior_n ≥ 10일 때만), $pb_prior_shr30 = (prior_s + 30 \cdot base) / (prior_n + 30)$ (base = p_prior_s_rate NaN → 0.52), $pb_prior_dev = pb_prior_shr30 - base$. 한 번도 만난 적 없으면 n=0, rate NaN, shr30 = base, dev 0. (원 구현은 입력 행 전체에 학습 매치업이 하나도 없을 때만 별도 분기로 pb_prior_shr30을 만들지 않고 dev를 NaN으로 두는데, 실제 테스트·샘플에서는 발생하지 않는다. 위 규칙대로 구현하면 전체 실행 결과와 같다.)
- LG_game = (pitcher_team_id==13 or batter_team_id==13) (0/1 실수); LG_regime = LG_game × [season > 2023 or (season == 2023 and (game_month ≥ 5 or game_type == 'F'))]. 즉 팀 13 관여 경기 중 2023년 5월 이후(퓨처스는 2023년 시즌 시작부터)의 "새 라벨 체제" 지시자. 이 규칙은 학습 라벨의 월별 성공률 편차(2023년 4월 -0.022 → 5월 +0.047)에서 발견한 것이다.
- 역할(role 표를 games를 가중치로 § 8.7의 가중 prior로 집계): $ROLE_games_prior = prior_n$ (이전 시즌 경기 수 합, 없으면 0), $ROLE_start_share = prior_starts$, $ROLE_ppg = prior_ppg$, $ROLE_ppg_last = last_ppg$, $ROLE_inn_first = prior_inn_first$, $ROLE_inn_last = prior_inn_last$, $ROLE_cur_pace = p_cur_n / \max(game_month - 2, 1)$, $ROLE_inning_minus_first = inning - ROLE_inn_first$.
- Trackman(tm 표를 tm_n 가중): $TM_n_prior = prior_n$ (없으면 0); 33개 지표 m(tm 표의 tm_열 중 tm_n 제외)마다 $TM_<m>에서 'tm_' 제거 = prior_m$ (예: TM_rel_speed_mean, TM_mix_Fastball, TM_fb_speed, TM_rel_consistency_h); $TM_last_<m> 8개 = 마지막 활동 시즌 값(fb_speed, rel_speed_mean, spin_rate_mean, fb_relh_std, fb_rels_std, rel_consistency_h, rel_consistency_s, minor_share)$; $TM_fb_speed_trend = TM_last_fb_speed - TM_fb_speed$.

8.7 가중 prior (역할·Trackman용)

(개체, 시즌) 표에서 가중치 열 w(=games 또는 tm_n)와 지표 m들에 대해, 시즌 격자(2019...2026)에서 $prior_n = \sum_{k < y} w_k$, $prior_m = \sum_{k < y, m_k \text{ 존재}} w_k \cdot m_k / \sum_{k < y, m_k \text{ 존재}} w_k$ (분모 0이면 NaN), $last_m = y$ 이전 활동(w>0) 시즌들 중 **m이 NaN이 아닌 가장 최근 값**(활동 시즌이라도 m이 NaN이면 그 전 시즌의 값을 이어받음; 원 구현의 forward-fill 동작). 예: 마지막 Trackman 시즌에 패스트볼이 없어 tm_fb_speed가 NaN인 투수는 그 이전 시즌의 값이 $TM_last_fb_speed$ 가 되고 $TM_fb_speed_trend$ 도 그 값으로 계산된다(해당 투수 11~12명).

```
def prior_weighted(tab, id_col, n_col, metrics, max_season=2026):
    ids = tab[id_col].unique(); seasons = np.arange(tab.season.min(), max_season + 1)
    t = tab.set_index([id_col, 'season']).reindex(pd.MultiIndex.from_product([ids, seasons], names=[id_col, 'season']))
    n = t[n_col].fillna(0.0).unstack('season'); res = {'prior_n': n.cumsum(axis=1).shift(1, axis=1).fillna(0.0)}; act = n.gt(0)
    for m in metrics:
        v = t[m].unstack('season')
        wsum = (v.fillna(0.0) * n).cumsum(axis=1).shift(1, axis=1).fillna(0.0); nn = (n * v.notna()).cumsum(axis=1).shift(1, axis=1).fillna(0.0)
        res['prior_' + m] = wsum / nn.replace(0, np.nan); res['last_' + m] = v.where(act).ffill(axis=1).shift(1, axis=1)
    return pd.concat({k: v.stack(dropna=False) for k, v in res.items()}, axis=1).reset_index()
```

8.8 변화(드리프트)·폼 피쳐

$p_chg_s = p_cur_s_rate - p_prior_s_rate$, $p_chg_s_last = p_cur_s_rate - p_last_rate$, $b_chg_s = b_cur_s_rate - b_prior_s_rate$, $b_chg_s_last = b_cur_s_rate - b_last_rate$, $p_chg_rev = p_cur_rev_rate - p_prior_rev_rate$, p_chg_mid , b_chg_mid 동일; $wp = p_cur_n / (p_cur_n + 200)$, $wb = b_cur_n / (b_cur_n + 200)$, $drift_est = (nan_to_num(p_chg_s) \cdot wp + nan_to_num(b_chg_s) \cdot wb) / (wp + wb + 1e-6)$; $p_cur_logn = \log1p(p_cur_n)$, $b_cur_logn = \log1p(b_cur_n)$; $p_form1 = asof_p_prev1_game_success_rate - p_cur_s_rate$, p_form3 , p_form5 동일($prev3/prev5$), $p_form5_prior = asof_p_prev5_game_success_rate - p_prior_s_rate$, $p_form_mid5 = asof_p_prev5_game_middle_rate - p_cur_mid_rate$, $p_form_trend = asof_p_prev1_game_success_rate - asof_p_prev5_game_success_rate$.

8.9 계산되지만 v48 모델이 쓰지 않는 피쳐 (구현 생략 가능)

$ps2/bs2$ 기반 잔차 스킴(p_skill^* , B_skill^*), L_all_ref , $p_cur_dev^*$, $b_cur_dev^*$, p_est , b_est , p_prior_logn , b_prior_logn . 단 L_ref (§ 8.10)는 추론의 LightGBM 경로에서 쓰이므로 필요하다. $home_team$ 은 NN 범주형으로만 쓰인다.

8.10 레벨 참조 L_ref

행의 $game_type$ g 에 대해 $L_ref = L_gt[season-1, g]$; $season$ 이 2019이면 $L_gt[2019, g]$; $season > 2024$ (테스트)이면 $L_gt[2024, g]$ (R 0.48971, F 0.45928).

8.11 최종 입력 목록

트리(LightGBM·CatBoost) 178개, 이 순서: $season$, $game_month$, $game_dow$, $inning$, is_top , $game_type$, F , $balls$, $strikes$, $count_state$, $outs$, run_top , run_bot , run_total , $score_diff_home$, $score_diff_p$, abs_score_diff , $r1$, $r2$, $r3$, $n_runners$, $base_state$, $home_we$, $pitcher_we$, li , $pitcher_hand$, $batter_hand$, $platoon_same$, $pitcher_team$, $batter_team$, $pitcher_id$, $batter_id$, $asof_p_n$, $asof_p_success_rate$, $asof_p_reverse_rate$, $asof_p_middle_rate$, $asof_p_ball_rate$, $asof_p_strike_rate$, $asof_p_fastball_rate$, $asof_p_breaking_rate$, $asof_p_offspeed_rate$, $asof_p_prev1_game_success_rate$, $asof_p_prev3_game_success_rate$, $asof_p_prev5_game_success_rate$, $asof_p_prev1_game_middle_rate$, $asof_p_prev3_game_middle_rate$, $asof_p_prev5_game_middle_rate$, $asof_b_n$, $asof_b_success_rate$, $asof_b_middle_rate$, p_prior_n , p_cur_n , $p_cur_s_rate$, $p_prior_s_rate$, $p_cur_rev_rate$, $p_prior_rev_rate$, $p_cur_mid_rate$, $p_prior_mid_rate$, $p_cur_ball_rate$, $p_prior_ball_rate$, $p_cur_strike_rate$, $p_prior_strike_rate$, $p_cur_fb_rate$, $p_prior_fb_rate$, $p_cur_br_rate$, $p_prior_br_rate$, $p_cur_os_rate$, $p_prior_os_rate$, p_cur_s , $p_prior_F_share$, $p_prior_F_rate$, $p_prior_F23_n$, $p_prior_F23_rate$, $p_prior_F23_share$, $p_prior_R_rate$, $p_prior_vsL_rate$, $p_prior_vsR_rate$, $p_prior_vsHand_rate$, CTX_hand_diff , CTX_hand_adj , CTX_2k_diff , CTX_2k_adj , p_last_n , p_last_rate , p_last_gap , $p_n_seasons$, $p_prior_srate_fb$, $p_prior_srate_br$, $p_prior_srate_os$, $p_mix_exp_rate$, $p_mix_exp_minus_prior$, b_prior_n , b_cur_n , $b_cur_s_rate$, $b_prior_s_rate$, $b_cur_mid_rate$, $b_prior_mid_rate$, b_last_rate , b_last_n , b_last_gap , $b_n_seasons$, $p_cur_s_shr100$, $p_cur_s_shr500$, $p_all_s_shr200$, $b_cur_s_shr300$, $b_all_s_shr300$, pb_prior_n , pb_prior_rate , pb_prior_shr30 , pb_prior_dev , LG_game , LG_regime , $ROLE_games_prior$, $ROLE_start_share$, $ROLE_ppg$, $ROLE_ppg_last$, $ROLE_inn_first$, $ROLE_inn_last$, $ROLE_cur_pace$, $ROLE_inning_minus_first$, TM_n_prior , $TM_rel_speed_mean$, $TM_rel_speed_std$, $TM_spin_rate_mean$, $TM_spin_rate_std$, $TM_induced_vert_break_mean$, $TM_induced_vert_break_std$, $TM_horz_break_mean$, $TM_horz_break_std$, $TM_extension_mean$, $TM_extension_std$, $TM_rel_height_mean$, TM

_rel_height_std, TM_rel_side_mean, TM_rel_side_std, TM_zone_speed_mean, TM_zone_speed_std, TM_mix_Fastball, TM_mix_Slider, TM_mix_Curveball, TM_mix_ChangeUp, TM_mix_Splitter, TM_mix_Sinker, TM_mix_Cutter, TM_minor_share, TM_fb_speed, TM_fb_spin, TM_fb_ivb, TM_fb_hb, TM_fb_relh_std, TM_fb_rels_std, TM_fb_speed_std, TM_rel_consistency_h, TM_rel_consistency_s, TM_last_fb_speed, TM_last_rel_speed_mean, TM_last_spin_rate_mean, TM_last_fb_relh_std, TM_last_fb_rels_std, TM_last_rel_consistency_h, TM_last_rel_consistency_s, TM_last_minor_share, TM_fb_speed_trend, p_cur_logn, b_cur_logn, p_chg_s, p_chg_s_last, b_chg_s, b_chg_s_last, p_chg_rev, p_chg_mid, b_chg_mid, drift_est, p_form1, p_form3, p_form5, p_form5_prior, p_form_mid5, p_form_trend

NN 수치 117개 = 위 178개에서 season, pitcher_team, batter_team, pitcher_id, batter_id, count_state, base_state, ROLE_* 8개, TM_* 43개(=1+33+8+1), CTX_* 4개를 제외한 116개(순서는 위 순서 유지) + 마지막에 season_rel = clip(2024 - season, 0, 10) (학습 행: 2024→0 ... 2019→5; 테스트 2025→0). NN 범주형(ID 모델 7개, 이 순서): pitcher_id, batter_id, pitcher_team, batter_team, home_team, count_state, base_state; 무ID 모델은 앞의 두 개를 뺀 5개.

9. [4단계] 투수 레벨의 동적 베이지 추정과 4클래스 오프셋

세 멤버(LightGBM, NN 2종)는 **오프셋(초기 점수)** 위에서 학습·예측한다. 오프셋은 각 행의 투수가 "현재 시즌에 어느 수준인지"를 상태공간 모형으로 추정한 성공 확률 pm에서 만든다. CatBoost 멤버는 오프셋을 쓰지 않는다.

9.1 시즌 간 재귀 (투수별 사전분포)

상수: $a = 0.6$, $\tau^2 = 0.004$, $v_{\text{new}} = 0.004$, 관측 분산 $\text{SIG}^2 = 0.25$. 시즌 격자 2019...2026. 시즌별 리그 평균 $\mu[y] = \text{league_rate}[y]$ (2025·2026은 2024 값 0.4861로 지속). 투수별 상태 (m, v) — 리그 평균 대비 편차의 평균·분산. 초기 $m=0$, $v=v_{\text{new}}$, started=False. 시즌 y를 차례로:

```
prior_m[y] = m ; prior_v[y] = v # 시즌 y를 보기 전의 사전분포 (저장)
n = ps[y].n, s = ps[y].s (해당 시즌 기록 없으면 n=0)
prec = 1/v + (n/SIG2 if n>0 else 0)
mean_obs = (s - n·mu[y]) / max(n,1) if n>0 else 0
m_post = (m/v + (mean_obs·n/SIG2 if n>0 else 0)) / prec ; v_post = 1/prec
started = started or (n>0)
다음 시즌으로: started이면 m = a·m_post, v = a²·v_post + tau2 ; 아니면 m = 0, v = v_new
```

9.2 행 사후분포

행의 (pitcher_id, season)으로 prior_m(없으면 0), prior_v(없으면 v_new)를 붙이고, $\mu_{\text{row}} = \mu[\text{season}]$, $n = \max(p_{\text{cur_n}}, 0)$, $s = \text{clip}(\text{nan_to_num}(p_{\text{cur_s}}), 0, n)$:

```
prec = 1/prior_v + n/SIG2
m1 = (prior_m/prior_v + (s - n·mu_row)/SIG2) / prec
bayes_dev = m1 # (원 코드: (mu_row + m1) - mu_row)
```

9.3 오프셋 벡터

$\text{pm} = \text{clip}(\mu_{\text{row}} + \text{bayes_dev} + \text{ctx}, 0.05, 0.95)$, 여기서 $\text{ctx} = \text{CTX_hand_adj} + \text{CTX_2k_adj}$ 는 **NN-ID 멤버만** 더하고(LightGBM·NN-무ID는 0). 4클래스 로그-사전확률: $\text{init} = [\log(\text{pm}), \log((1-\text{pm}) \cdot \text{fshare}[0])]$,

$\log((1-pm) \cdot fshare[1]), \log((1-pm) \cdot fshare[2])]$, $fshare = [0.4810751, 0.2424522, 0.2764727]$

(§ 4.2). 학습 행의 오프셋은 학습 행 자신의 (시즌, p_cur_n, p_cur_s)로, 테스트 행은 2025 사전분포(2024 시즌 말 상태에 한 번 전이)와 행의 시즌 내 기록으로 계산한다.

```
def season_priors(ps, league, a=0.6, tau2=0.004, v_new=0.004, max_season=2026, SIG2=0.25):
    seasons = np.arange(int(ps.season.min()), max_season + 1); pit = np.sort(ps.pitcher_id.unique())
    n_w = ps.pivot(index='pitcher_id', columns='season', values='n').reindex(index=pit, columns=seasons).fillna(0.0).values
    s_w = ps.pivot(index='pitcher_id', columns='season', values='s').reindex(index=pit, columns=seasons).fillna(0.0).values
    mu = np.array([league.get(int(y), np.nan) for y in seasons]); last = mu[~np.isnan(mu)][-1]; mu = np.where(np.isnan(mu), last, mu)
    m = np.zeros(len(pit)); v = np.full(len(pit), v_new); started = np.zeros(len(pit), bool); pm_ = np.zeros_like(n_w); pv_ = np.zeros_like(n_w)
    for j, y in enumerate(seasons):
        pm_[:, j] = m; pv_[:, j] = v; n = n_w[:, j]; s = s_w[:, j]; obs = n > 0
        prec = 1.0 / v + np.where(obs, n / SIG2, 0.0); mean_obs = np.where(obs, (s - n * mu[j]) / np.maximum(n, 1), 0.0)
        m_post = (m / v + np.where(obs, mean_obs * n / SIG2, 0.0)) / prec; v_post = 1.0 / prec; started |= obs
        m = np.where(started, a * m_post, 0.0); v = np.where(started, a * a * v_post + tau2, v_new)
    tab = pd.DataFrame({'pitcher_id': np.repeat(pit, len(seasons)), 'season': np.tile(seasons, len(pit)), 'prior_m': pm_.ravel(), 'prior_v': pv_.ravel()})
    return tab, dict(zip(seasons.tolist(), mu.tolist()))
def offset_init(df, F, ps, league, fshare, ctx=None, v_new=0.004, SIG2=0.25):
    tab, mu = season_priors(ps[['pitcher_id', 'season', 'n', 's']], league)
    d = df[['pitcher_id', 'season']].reset_index(drop=True).merge(tab, on=['pitcher_id', 'season'], how='left')
    m0 = d.prior_m.fillna(0.0).values; v0 = d.prior_v.fillna(v_new).values; mu_row = pd.Series(df.season.values).map(mu).values
    n = np.maximum(F['p_cur_n'].values, 0.0); s = np.clip(np.nan_to_num(F['p_cur_s'].values), 0.0, n)
    prec = 1.0 / v0 + n / SIG2; m1 = (m0 / v0 + (s - n * mu_row) / SIG2) / prec
    pm = np.clip(mu_row + m1 + (0.0 if ctx is None else ctx), 0.05, 0.95)
    return np.stack([np.log(pm)] + [np.log((1 - pm) * fshare[k]) for k in range(3)], 1)
```

10. [5단계] 모델 학습 (전부 2019~2024 전체 행, 4클래스 라벨 y4)

10.1 공통 준비

- 피쳐 행렬 F (§ 8), 라벨 y_4 (§ 4.2), 오프셋 $init$ (§ 9.3; 멤버별 ctx 여부 다름).
- 최근 시즌 가중치 $w_i = \exp(-0.15 \times (2024 - season_i))$ (2024:1.0, 2023:0.861, 2022:0.741, 2021:0.638, 2020:0.549, 2019:0.472) — LightGBM과 NN-ID에만 적용. 퓨처스 행 $\times 2$ 가중 — LightGBM에만 적용($w_i \times 2$).
- 학습 순서: [1] 대응표 $\rightarrow$ [2] 통계 $\rightarrow$ [3][4] 피쳐·오프셋 $\rightarrow$ LightGBM(CPU)과 CatBoost(GPU)는 동시에 실행 가능 $\rightarrow$ NN-ID $\rightarrow$ NN-무ID $\rightarrow$ 앙상블 설정 $\rightarrow$ 패키징. 멤버 간 의존성은 없다(모두 같은 F 와 $init$ 를 사용).

10.2 멤버 A — LightGBM 4클래스 + 베이지 오프셋 (lgb_mc_off_ctx_lgm_fw2, 블렌드 가중 0.25)

- 입력 178개 (§ 8.11), categorical_feature = [pitcher_team, batter_team, base_state, count_state], init_score = init(n×4), weight = w_i × (2 if game_type=='F' else 1).
- 파라미터(그 외는 LightGBM 4.7.0 기본값): objective multiclass, num_class 4, boosting gbd, learning_rate 0.03, num_leaves 63, min_data_in_leaf 500, feature_fraction 0.7, bagging_fraction 0.8, bagging_freq 1, lambda_l2 10.0, num_threads 8, verbose -1, max_depth -1, min_sum_hessian_in_leaf 0.001, max_bin 255, deterministic false. 라운드 180(→ 트리 720개 = 180×4클래스). 범주형 관련 기본값(값에 직접 영향): min_data_per_group 100, cat_smooth 10, cat_l2 10, max_cat_to_onehot 4, max_cat_threshold 32; 구간화 기본값 max_bin 255, bin_construct_sample_cnt 200000, min_data_in_bin 3, use_missing true, zero_as_missing false; 범주형 열 인덱스 [8, 20, 27, 28], pandas 범주형 dtype 미사용(정수 코드).
- 시드 8개: $s \in \{0, \dots, 7\}$ 마다 seed = bagging_seed = feature_fraction_seed = s. 8개 부스터를 각각 저장.
- 체크포인트: 학습 행 앞 200,000개에 대해 init 없이 predict 한 클래스 0 확률 평균 ≈ 0.243 (시드별 0.2428~0.2431; init을 빼고 계산하므로 0.5가 아님이 정상). 같은 스레드 수·버전이면 모델 파일이 비트 단위로 재현된다.

10.3 멤버 B — CatBoost 4클래스 + ID 범주형 (cat_mc_ids3_ctx_lgm, 가중 0.35)

- 입력 178개, 범주형 = [pitcher_team, batter_team, base_state, count_state, pitcher_id, batter_id] (정수 캐스팅), 오프셋·가중치 없음.
- 파라미터: loss_function MultiClass, iterations 2750, learning_rate 0.02, depth 7, l2_leaf_reg 10, border_count 128, task_type GPU, thread_count 8, random_seed s (0~7), verbose 0. 이때 CatBoost 1.2.10 GPU 기본값으로 확정되는 값: boosting_type Plain, bootstrap_type Bayesian, bagging_temperature 1, random_strength 1, one_hot_max_size 2, max_ctr_complexity 4, grow_policy SymmetricTree, min_data_in_leaf 1, leaf_estimation_method Newton, leaf_estimation_iterations 1, feature_border_type GreedyLogSum, score_function Cosine, nan_mode Min, model_size_reg 0.5, rsm 1, fold_permutation_block 64, permutation_count 4, simple_ctr [Borders(CtrBorderCount 15, Uniform, TargetBorderCount 1, MinEntropy, Prior 0/1·0.5/1·1/1), FeatureFreq(CtrBorderCount 15, MinEntropy, Prior 0/1)], combinations_ctr [Borders(동일), FeatureFreq(CtrBorderCount 15, Median, Prior 0/1)], counter_calc_method SkipTest.
- 저장된 모델에 함께 기록된 그 밖의 기본값: boost_from_average False, max_leaves 128, random_score_type NormalWithModelSizeDecrease, leaf_estimation_backtracking AnyImprovement, bayesian_matrix_reg 0.1, ctr_history_unit Sample, min_fold_size 100, observations_to_bootstrap TestOnly, has_time False, eval_metric MultiClass. 범주형 열 인덱스 [8, 20, 27, 28, 29, 30], 트리 2,750개.
- 체크포인트: 학습 행 앞 200,000개 predict_proba 클래스 0 평균 ≈ 0.551 (시드별 0.5507~0.5510). 시드당 약 2.7분(RTX 3060).

10.4 멤버 C·D — 신경망 (nn_ids_off8w_ctx_lgm 가중 0.30 · nn_wide3_off_lgm 가중 0.10)

입력 변환(학습 행으로 fit, 그대로 저장해 추론에 사용): - 수치 117개 행렬 X(float32). QuantileTransformer(n_quantiles=200, output_distribution='normal', subsample=200000, random_state=0)를 nan_to_num(X, 0)으로 fit(scikit-learn 1.8.0 — 이 구현은 열마다 RandomState(0)에서 20만 행을 순차 추출하므로 같은 라이브러리 버전을 써야 분위수가 같아진다). 변환 후 원래 NaN 자리는 0. 결측 지시자: 학습에서 NaN 비율 > 0.1%인

열(58개: asof_p_prev* 6개, p_cur*_rate/p_prior*_rate 16개, p_prior_F_share, p_prior_F_rate, p_prior_F23_rate, p_prior_F23_share, p_prior_R_rate, p_prior_vsL_rate, p_prior_vsR_rate, p_prior_vshand_rate, p_last_n, p_last_rate, p_last_gap, p_prior_srate_fb/br/os, p_mix_exp_minus_prior, b_cur_s_rate, b_prior_s_rate, b_cur_mid_rate, b_prior_mid_rate, b_last_rate, b_last_n, b_last_gap, pb_prior_rate, p_chg_s, p_chg_s_last, b_chg_s, b_chg_s_last, p_chg_rev, p_chg_mid, b_chg_mid, p_form1, p_form3, p_form5, p_form5_prior, p_form_mid5, p_form_trend)의 NaN 여부(0/1)를 뒤에 붙여 입력 차원 117+58 = 175. - 범주형: 각 열의 값을 **학습 행에서 처음 등장한 순서대로 1, 2, ...**로 코드화(0 = 미지). 임베딩 크기(고유값 수+1, 차원): pitcher_id (792+1, 64), batter_id (830+1, 32), pitcher_team (13+1, 4), batter_team (13+1, 4), home_team (12+1, 4), count_state (12+1, 4), base_state (8+1, 3). 무ID 모델은 뒤 5개만. **구조:** 입력 = [수치 175 || 임베딩 연결] → Linear(→768) → BatchNorm1d → SiLU → Dropout(0.2) → Linear(→384) → BN → SiLU → Dropout → Linear(→192) → BN → SiLU → Dropout → 세 개의 출력층: 4클래스(주), 볼/스트라이크/기타 3클래스, 구종군 3클래스. 초기화는 PyTorch 기본. **학습:** 시드마다 torch.manual_seed(seed); np.random.seed(seed); AdamW(lr=1e-3, weight_decay=1e-5); OneCycleLR(max_lr=1e-3, total_steps=8×ceil(1,475,092/4096)=8×361=2,888, pct_start=0.1); 에폭 8, 배치 4096, 에폭마다 torch.randperm으로 셔플; 주 4클래스 로짓에 오프셋 init4를 **더한 뒤** 교차엔트로피; 손실 = CE(4클래스) + 0.5×[CE(볼/스트라이크) + CE(구종군)]. ID 모델은 배치마다 pitcher_id/batter_id 코드를 각각 확률 0.15로 0(미지)으로 치환(ID 드롭아웃). 학습 후 state_dict를 CPU로 저장(BatchNorm 러닝 통계 포함); 추론 시 eval() 모드로 사용. 명시하지 않은 값은 PyTorch 2.11 기본값: AdamW betas (0.9, 0.999)·eps 1e-8, OneCycleLR anneal_strategy cos·div_factor 25·final_div_factor 1e4·three_phase False·**cycle_momentum True(β1이 0.95↔0.85로 순환)**, BatchNorm1d eps 1e-5·momentum 0.1, 마지막 배치 532행. - NN-ID: 시드 0~7(8개), 가중치 $w_i = \exp(-0.15(2024 - \text{season}))$ 로 가중 평균 손실(세 항 각각 $\sum w \cdot \text{CE} / \sum w$), 오프셋 = 베이즈 + CTX (§ 9.3). - NN-무ID: 시드 0,1,2(3개), 가중치 없음(단순 평균 손실), 오프셋 = 베이즈만, 범주형 5개. - 체크포인트: 에폭 평균 손실(가중) NN-ID 시드0: 2.169 → 2.109 → 2.101 → 2.096 → 2.092 → 2.089 → 2.086 → 2.084 (다른 시드도 마지막 2.084±0.001); NN-무ID 시드0: 2.158 → 2.104 → 2.098 → 2.096 → 2.093 → 2.091 → 2.089 → 2.088. 시드당 약 25초(RTX 3060).

11. [6단계] 추론 — 테스트 행의 예측 생성

11.1 입력

./data/test.csv, ./data/sample_submission.csv; 학습 산출물: 통계 표 10개, 부스터 파일(LightGBM 8, CatBoost 8, NN 11), NN 변환기(분위수 변환기·결측열 목록·범주 코드표·임베딩 크기), 블렌드 설정. 피쳐 F (§ 8)와 오프셋 (§ 9)을 테스트 행에 대해 계산한다. 테스트 시즌 2025에 대한 참조값: $L_{\text{ref}} = L_{\text{gt}}[2024, g]$, $\mu_{\text{row}} = 0.4861$, $\text{season}_{\text{rel}} = 0$, 이전 시즌 합계 = 2019~2024 전체.

11.2 멤버별 예측 (모두 "성공 확률"로 통일)

- **LightGBM:** 부스터마다 $z = \text{raw_score}(178\text{피쳐}) + \text{init} (n \times 4) \rightarrow p_b = \text{softmax}(z)[:, 0]$; $\text{raw} = \text{mean}_b \text{logit}(\text{clip}(p_b, 1e-6, 1-1e-6))$ (8개 부스터의 logit 평균); **그다음** $\text{raw} += \text{logit}(L_{\text{ref}})$ (R행 $\text{logit}(0.48971) = -0.0412$, F행 $\text{logit}(0.45928) = -0.1633$) — 이 항은 학습 시 오프셋에 포함되지 않았던 값이 추론에서만 더해지는 **구현상 특이점**이지만 v48의 실제 예측에 포함되어 있으므로 반드시 그대로 재현한다; $p_A = \text{sigmoid}(\text{raw})$.
- **CatBoost:** 부스터마다 $\text{predict_proba}(178\text{피쳐}, \text{범주형 } 6\text{개})$ 의 클래스 0 확률 → logit → 8개 평균 → $\text{sigmoid} = p_B$. (오프셋 없음.)

- **NN:** 수치 175 벡터·범주 코드를 만들고, 부스터(시드)마다 `softmax(head4 + init4)[: ,0]` 을 계산해 **확률을 시드 평균**(logit 평균이 아님) → `p_c`(무ID, 3시드), `p_d`(ID, 8시드; init4에 CTX 포함). 추론은 CPU, 배치 32,768.

11.3 블렌드와 후처리 (정확한 수치)

```
w = [0.25 (A: LightGBM), 0.35 (B: CatBoost), 0.10 (C: NN-무ID), 0.30 (D: NN-ID)] # 합 1
p = sigmoid( Σ_k w_k · logit(clip(p_k, 1e-6, 1-1e-6)) )
p = p - 0.004 # 전역 레벨 시프트
LG행 := game_type != 'F' 이고 (pitcher_team_id==13 또는 batter_team_id==13)
p = p + 0.015 (LG행) # 투수 측/타격 측 동일(lg_shift_p
= lg_shift_opp = 0.015)
p = p + 0.09 (LG행 이고 balls_before==3 이고 strikes_before==0) # 3-0 카운트 규칙
p = clip(p, 0.001, 0.999)
```

그 외 설정값 `f_shift 0`, `lg_shift_home 0`, `lg_shift_away 0`, `cold_slope 0`, `sharpen []` 은 모두 비활성(아무 변화 없음).

11.4 제출 파일

`sample_submission.csv`의 `row_id` 순서대로 `control_success = p[row_id]`; 매칭 실패 시 상수 0.4861(2024 리그 평균)로 채움(실제 평가에서는 발생하지 않음). `./output/submission.csv`에 `row_id,control_success 2` 열, 인덱스 없이 UTF-8로 저장.

11.5 추론 예시 (핵심 부분)

```
from scipy.special import expit, logit, softmax
p_A = expit(np.mean([logit(np.clip(softmax(b.predict(X, raw_score=True) + init, axis=1)[: , 0], 1e-6, 1-1e-6)) for b in lgb_boosters], 0) + logit(L_ref))
p_B = expit(np.mean([logit(np.clip(m.predict_proba(Xc)[: , 0], 1e-6, 1-1e-6)) for m in cat_models], 0))
def nn_prob(nets, Xn, Xc, init4): # 확률 평균
    return np.mean([torch.softmax(net(torch.tensor(Xn), torch.tensor(Xc)))[0] + torch.tensor(init4), 1)[: , 0].detach().numpy() for net in nets], 0)
p_C = nn_prob(nets_noid, Xn, Xc5, init4_bayes); p_D = nn_prob(nets_id, Xn, Xc7, init4_bayes_ctx)
p = expit(0.25*logit(np.clip(p_A, 1e-6, 1-1e-6)) + 0.35*logit(np.clip(p_B, 1e-6, 1-1e-6)) + 0.10*logit(np.clip(p_C, 1e-6, 1-1e-6)) + 0.30*logit(np.clip(p_D, 1e-6, 1-1e-6)))
p = p - 0.004
is_lg = (test.game_type.values != 'F') & ((test.pitcher_team_id.values == 13) | (test.batter_team_id.values == 13))
p = p + 0.015 * is_lg + 0.09 * (is_lg & (test.balls_before.values == 3) & (test.strikes_before.values == 0))
p = np.clip(p, 0.001, 0.999)
```

12. 처음부터 끝까지 실행 순서와 환경

12.1 환경 (학습·추론 모두)

Python 3.11.15; numpy 1.26.4; pandas 2.0.3; scipy 1.15.3; scikit-learn 1.8.0; joblib 1.5.3; pyarrow 25.0.1(선택, CSV 캐시용); lightgbm 4.7.0; catboost 1.2.10(GPU 빌드); torch 2.11.0+cu128. 학습 하드웨어

기준: RTX 3060 12GB + 16 CPU. 평가 서버(추론)는 lightgbm==4.7.0, catboost==1.2.10만 requirements.txt로 설치하고 나머지(numpy/pandas/scipy/joblib/torch)는 기본 설치본을 쓴다; 추론은 CPU만으로 32초.

12.2 순서

- 1. train.csv / trackman_history.csv 로드 (utf-8-sig). 파일 순서 유지.
- 2. § 6 Trackman 대응표 → 투수 760명 (체크: 유사도>0.9 경기 87.3%).
- 3. § 7 통계 표 10종 (체크: ps 2,260행·bs 2,393행·pbs 150,624행·role 2,260행·tm 2,305행; ps.n 합 1,475,092).
- 4. § 8 학습 행 피쳐 F (178열 + NN용 117열 + L_ref), § 4 라벨 y4/보조 라벨, § 9 오프셋 init(베이스), init_ctx(베이스+CTX).
- 5. § 10.2 LightGBM 8시드 (CPU 8스레드, 약 12분) || § 10.3 CatBoost 8시드 (GPU, 약 21분).
- 6. § 10.4 NN-ID 8시드(약 4분) → NN-무ID 3시드(약 1.5분). 분위수 변환기·범주 코드표 저장.
- 7. 블렌드 설정 저장: 가중 .25/.35/.10/.30, logit 블렌드, prob_shift -0.004, lg_shift 0.015 (p/opp 동일), lg30_shift 0.09, f_shift 0, home/away 0.
- 8. 추론 스크립트 + model/ 폴더 + requirements.txt (lightgbm==4.7.0, catboost==1.2.10)를 zip으로 패키징 (총 37파일, ≈87 MB).
- 9. § 11 추론: test.csv → submission.csv.

12.3 예상 중간 산출물

산출물	형상/크기	확인값
투수 대응표	760행 (pitcher_id, tm_id, votes, purity)	purity>0.9: 99.6%
통계 표	ps 2,260×32 · bs 2,393×7 · pbs 150,624×4 · league 6×3 · L_gt 6×2 · L_all 6 · ps2 2,260×23 · bs2 2,393×9 · role 2,260×7 · tm 2,305×36	§ 1.4, § 7.1 합계
학습 피쳐	1,475,092 × 178 (+L_ref)	결측률: p_cur_s_rate 등 시즌 초 행에서 NaN
라벨	y4 분포 [772,603, 337,950, 170,320, 194,219]	fshare [.48108, .24245, .27647]
LightGBM	8 × 약 5.2 MB 텍스트, 720트리	시드 파일 비트 동일 재현
CatBoost	8 × 약 14.5 MB	첫 200k행 클래스0 평균 ≈ 0.551
NN	ID 모델 8 × 2,721,333 B, 무ID 모델 3 × 2,116,285 B, 변환기 2개	예폭 손실 § 10.4
추론	253,507행, 32초	§ 13

13. 재현 성공 판정 체크포인트 (테스트 253,507행 기준)

항목	v48 값
멤버별 테스트 평균 예측	LightGBM 0.4898 · CatBoost 0.4708 · NN-무ID 0.5366 · NN-ID 0.5268
최종 평균 / 표준편차	0.4979 / 0.0404 (F행 0.4652, R행 0.5022)
최종 분위수 (0, 1%, 10%, 50%, 90%, 99%, 100%)	0.3511, 0.4078, 0.4485, 0.4960, 0.5515, 0.5932, 0.7449

항목	v48 값
배포 test.csv 5행 샘플(TEST_000001, 000017, 000213, 005332, 035185)	0.4094, 0.3757, 0.4468, 0.4864, 0.5122
행 독립성	임의의 1행만 넣은 예측 = 전체를 넣었을 때의 그 행 예측 (차이 $< 1e-8$)
후처리 타깃 검증	3-0 규칙 전후 차이가 정확히 590행 +0.09, LG 시프트가 44,768행 +0.015
허용 오차	행당 평균 절대 차이 $\leq 1e-3$ (GPU 비결정성 수준 $2.5e-4$)

샘플 5행의 입력 요약(참조): TEST_000001 투수 21813 (asof_n 3465, 성공률 0.4895, R, 팀 16 vs 21, 1-0 카운트) → 0.4094; TEST_005332 투수 24713 (asof_n 0 = 통산 첫 투구, 성공률 NaN) → 0.4864(콜드스타트: 이전 시즌 통계도 없으면 리그 수준 근처).

14. v48 규칙 준수 감사 (대회 규칙 원문·주최측 답변 기준)

명확히 허용 - 추론은 각 행의 자기 컬럼 + 학습 데이터 통계만 사용(규칙 4항 그대로). 테스트 행 간 집계·분포·순서 정보 없음(단일행 = 전체 예측으로 실증). - asof_* 컬럼 사용과 "학습기간 누적을 빼서 현 시즌분 복원" (§ 8.3): 주최측 08-13 답변에서 명시적으로 허용. - 학습 데이터 안에서 다음 행을 참조한 라벨 복원 (§ 4.2)과 보조 라벨·보조 헤드: 08-10, 08-19, 08-20 답변 허용("학습 데이터 내에서는 제약 없음"). - Trackman 대응표 추정과 이전 시즌 Trackman 요약 피쳐 (§ 6, § 8.6): 08-07, 08-11 답변 허용(2019~2024 이력만). - 팀 13 관여·2023년 5월 체제 피쳐와 3-0 규칙 (§ 8.6, § 11.3): 행 자신의 팀 ID·월·카운트와 학습 라벨 통계에서만 도출(외부 자료 없음). 3-0 보정 +0.09는 학습(2024 검증) 잔차 +0.105에서 정한 값. - 사전학습 모델·외부 API·외부 데이터 없음; 오프라인 32초 추론; 제출 규격 준수.

해석상 애매함 - lg_shift +0.015의 크기: 학습 관찰로 방향·범위([0, +0.0164])를 정했지만 최종 크기는 같은 모델에 상수만 바꾼 리더보드 제출 3회(-0.005/+0.010/+0.015)를 비교해 골랐다. 주최측 08-12·08-19 답변("학습 범위 내 후보값 2~3회 비교·보간은 허용, 과도한 반복은 프로빙으로 검토 가능") 범위 안이지만, 코드 검증 시 산출 근거 소명 요청 대상이 될 수 있다. - prob_shift -0.004: 학습 검증(2024 폴드 평균 레벨)으로 근거를 갖고 초기 2회 리더보드 비교로 확정한 값. 위와 같은 성격이나 위험은 낮다.

사용하지 말아야 할 요소 (이 파이프라인을 확장·변형할 때 금지) - 테스트 행들의 통계(선수·팀·월별 누적, 빈도, 분포, target encoding, 행 순서 기반 rolling/expanding), 테스트 예측의 평균·분산을 이용한 사후 보정 — 규칙 4항·설명서 5항 위반. - 테스트의 다른 행 asof_*로 특정 행의 정답을 복원하거나 시간상 앞선 테스트 행을 참조하는 것 — 08-19 답변에서 명시적 위반. - 리더보드 점수 변화를 분석해 테스트 정답의 서브그룹 평균·분포를 역추정하는 반복 제출(상수만 바꾼 탐침) — 08-19·08-25 답변에서 "과도한 프로빙"으로 위반 검토 대상. (개발 중 준비했던 F/LG분할/비LG 탐침 제출물은 이 이유로 제출하지 않았다.) - 외부 데이터·지식(ABS 도입 시점, 구단 발표 등), 2025 Trackman, 현재 투구의 위치·판정·결과·구종·측정값.

15. 문서-코드 대조 감사 결과

작성 후, 이 문서를 실제 제출물의 추론 스크립트(script.py 1,017줄: 피쳐·오프셋·NN·main), 학습 스크립트(train_final.py, nn_export.py, trackman_match.py, save_stats.py, set_ensemble.py), 파라미터 정의(exp.py, exp2.py, nn_core.py), 실행 명령(reproduce.sh)과 실제 산출물(4개 meta.json, 2개 prep.pkl, stats.pkl,

LightGBM 모델 헤더, CatBoost 저장 파라미터, 대응표)에 대해 독립 검토자가 한 줄씩 대조했고, 체크포인트 수치는 데이터로 재계산했다.

대조 결과 (수정 반영 전 발견 → 모두 본문에 반영 완료) | # | 위치 | 발견 내용 | 심각도 | 조치 | |---|---|---|---|---|
1 | § 4.2 | 타자 라벨 복원의 마지막 행 mid 보정에서 성공률이 타자값이 아니라 상수 0.52 | 예측 영향(작음) | 명시 + 예시 호출 추가 | 2 | § 7.8 | tm_rel_consistency_*: std가 NaN인 구종군 그룹은 분자 0·분모 포함 (pandas sum 동작) — 2,305행 중 950행 영향 | 예측 영향 | 규칙 명시 | 3 | § 8.7 | last_m은 "마지막 활동 시즌 값"이 아니라 "이전 활동 시즌 중 마지막 non-NaN 값"(forward-fill) — 투수 11~12명 | 예측 영향(희소) | 규칙 명시 | 4 | § 7.8 | Trackman 지표 mean/std는 NaN 제외, tm_n·그룹 n은 NaN 행 포함 | 예측 영향(명세 누락) | 규칙 명시 | 5 | § 8.6, § 8.11 | TM 지표 수 35→33, TM_* 피쳐 수 52→43 (명시 목록 자체는 정확) | 서술 | 정정 | 6 | § 7.8 | 필터 후 Trackman 투수 701명, 중복 대응 제거됨 | 서술 | 체크포인트 추가 | 7 | § 12.3 | NN 무ID 파일 크기 2,116,285 B | 서술 | 정정 | 8 | § 10.4 | eval() 표현, PyTorch/sklearn 기본값 (OneCycleLR cycle_momentum 등) | 서술/보완 | 명시 | 9 | § 10.2-10.3 | LightGBM 범주형·구간화 기본값, CatBoost 저장 기본값 추가 | 보완 | 명시 | 10 | § 8.6 | 매치업 표가 전혀 없는 입력에서만 나타나는 원 구현의 별도 분기(실제 미발생) | 참고 | 주석 |

불일치가 없음을 확인한 부분: § 0 흐름·후처리 순서; § 1 데이터 사실(시즌별 행 수·성공률·L_gt 소수점까지); § 2 시간축(MAX_SEASON 2026, 2019 자기값, 2025→2024 지속); § 3 전 시즌 학습; § 4 y4/fshare/보조 라벨 분포(재계산 일치); § 5; § 6 매칭 알고리즘과 체크포인트 전부(경기 4,868, 유사도 분위수, 760/816명, purity); § 7 표 형상·합계; § 8.1-8.10 모든 수식·조건·NaN 규칙·상수(K=1000, 0.28, 0.5, 30, 100, 10, 20, 0.52, 200, 300 ...), 178개 목록·순서 = meta feats, NN 117·결측 지시자 58·범주 7/5·임베딩 크기 = prep.pkl; § 9 베이지 상수와 CTX 적용 대상(NN-ID만); § 10 가중치·F×2(LightGBM만)·CatBoost 무가중/무오프셋·NN 손실/ID 드롭아웃/시드/에폭/배치/스텝 2,888; § 11 LightGBM +logit(L_ref) 특이점, CatBoost 클래스0 logit 평균, NN 확률 평균, 블렌드 [0.25/0.35/0.10/0.30], -0.004, +0.015, +0.09, clip, 매칭 실패 상수 0.486105, 저장 형식; § 12 환경·env 변수·패키징(37파일); § 14 상수와 ensemble.json 일치.

결론: 수정 반영 후 문서는 v48 파이프라인을 계산 단위까지 완전히 기술하며, 남은 재현 오차 원인은 문서가 아니라 GPU 학습(CatBoost·NN)의 비결정성뿐이다(§ 0의 허용 오차 참조).